

Segmentation Model, Inference on ROI

ROI:



Width = 1010, height = 668
Model Input Patch Size = 512

Inference using Fully Connected Net

1) Pad the image:



```
import numpy as np

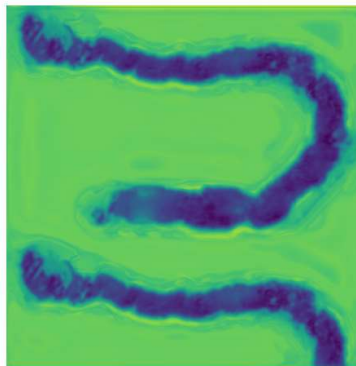
def pad_image(image, target_size):
    h, w, c = image.shape
    new_h = ((h + target_size - 1) // target_size) * target_size
    new_w = ((w + target_size - 1) // target_size) * target_size

    pad_h = new_h - h
    pad_w = new_w - w

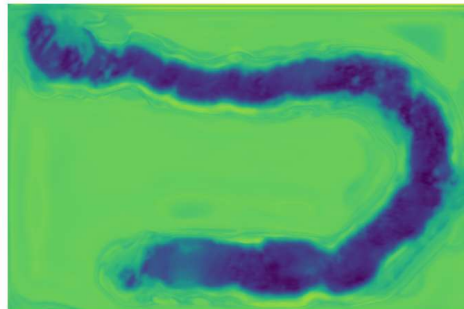
    padded = np.pad(image, ((0, pad_h), (0, pad_w), (0, 0)), mode='wrap')
    return padded.astype(np.uint8)
```

Width = height = 1024 (mode = Wrap)

2) Passing the padded image to the model (model output, logits):



3) Cropping the padded image output to the original image size:



4) Applying argmax to obtain the class prediction:

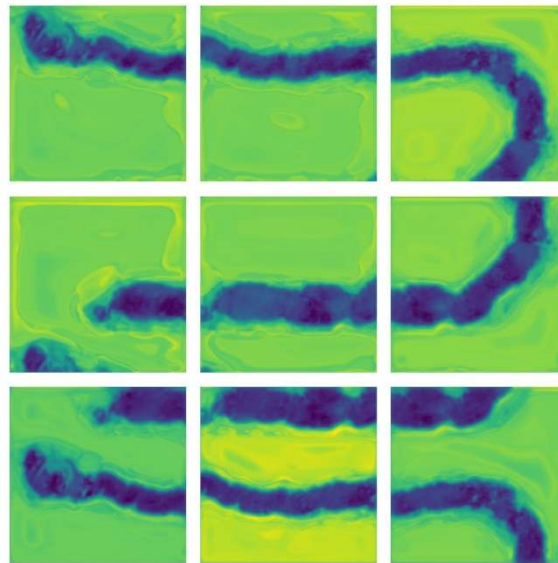


Inference using Sliding Window

- 1) Pad the image, same as before.
- 2) Extracting patches of size 512 with stride of 256 using the sliding window technique:



- 3) Passing the extracted patches to the model (model output, logits):



- 4) After running the sliding-window inference, each window produces a logits tensor of shape $(C, 512, 512)$, where C is the number of classes. Because the stride is 256, adjacent windows overlap by 50%, which means many pixels are predicted multiple times. To reconstruct the full-resolution segmentation map, we need to merge these overlapping predictions. The stitching process works as follows:
 - Initialize an empty logits canvas: Create an array of the size of the padded image, shaped (C, H, W) , along with a counter map to track how many times each pixel has been covered.
 - Accumulate logits from each patch: For every sliding window position, place its logits into the corresponding location in the canvas.
 - In overlapped regions, logits from multiple patches are *summed* (or averaged later), ensuring all predictions contribute.

```
import numpy as np

def stitch_predictions(image, preds, x_vals, y_vals):
    n_channels, patch_size, patch_size = preds[0].shape

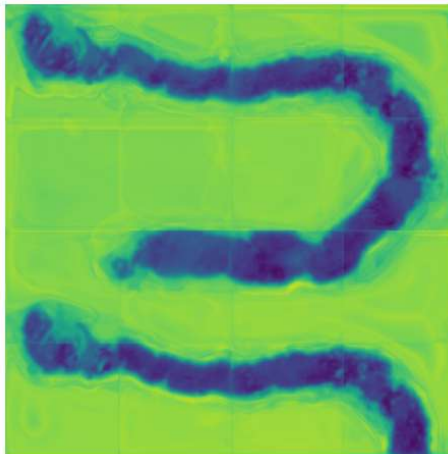
    stitched_image = np.zeros((n_channels, image.shape[0], image.shape[1]))
    counter = np.zeros((image.shape[0], image.shape[1]))

    idx = 0
    for y in y_vals:
        for x in x_vals:
            stitched_image[:, y : y + patch_size, x : x + patch_size] += preds[idx]
            counter[y : y + patch_size, x : x + patch_size] += 1
            idx += 1

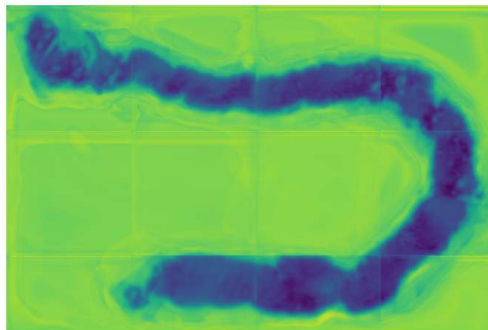
    counter[counter == 0] = 1

    stitched_image /= counter

    return stitched_image
```



5) Cropping the padded image output to the original image size:

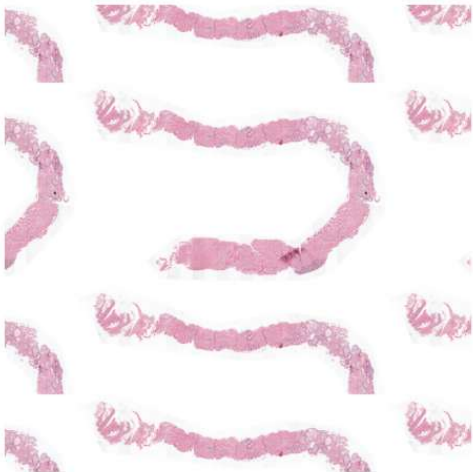


6) Applying argmax to obtain the class prediction:



Inference using Center Crop

- 1) Pad the image, same as before.
- 2) Pad the image again for center crop approach, adds half of the input patch size on both sides of padded image:



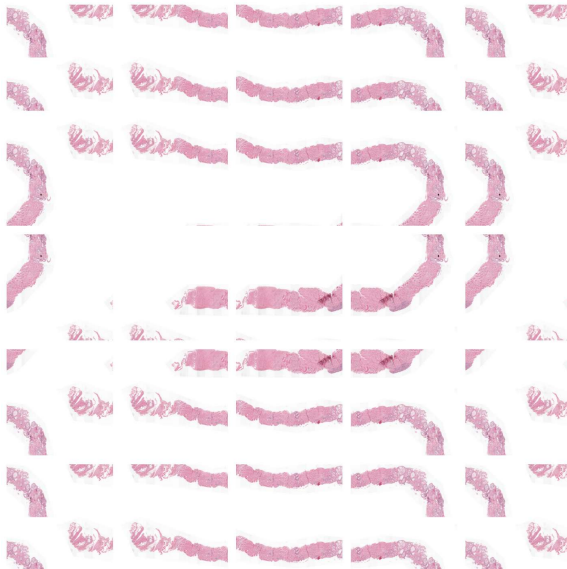
```
import numpy as np

def pad_image_center_crop(image, patch_size = 512):
    h, w, c = image.shape

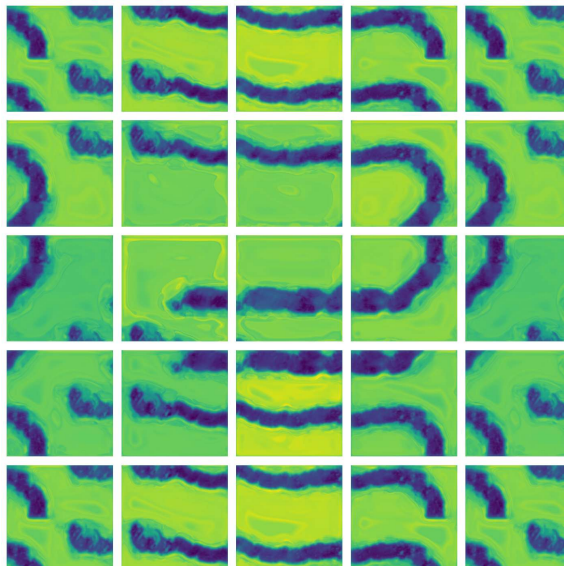
    pad_h = patch_size // 2
    pad_w = patch_size // 2

    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w), (0, 0)), mode='wrap')
    return padded.astype(np.uint8)
```

- 3) Extracting patches of size 512 with stride of 256 using the sliding window technique:



- 4) Passing the extracted patches to the model (model output, logits):



5) In the next step we only crop the center of each predicted mask and stitch them back:

```
import numpy as np

def stitch_predictions_center_crop(image, preds, x_vals, y_vals):
    n_channels, patch_size, patch_size = preds[0].shape

    step = patch_size // 4

    window = patch_size // 2

    stitched_image = np.zeros((n_channels, image.shape[0], image.shape[1]))

    idx = 0
    for y in y_vals:
        for x in x_vals:
            stitched_image[:, y + step: y + step + window,
                           x + step: x + step + window] = preds[idx][:, step: step + window,
                                                                           step: step + window]

            idx += 1

    return stitched_image
```

